Tutorial 08

▼ Overview

Tutorial (More More More Graphs)

- Directed Graphs
- Reachability within directed graphs
- Dijkstra's Algorithm
- Kruskal's Algorithm
- Prim's Algorithm
- Minimum Spanning Trees

Lab (From Page)

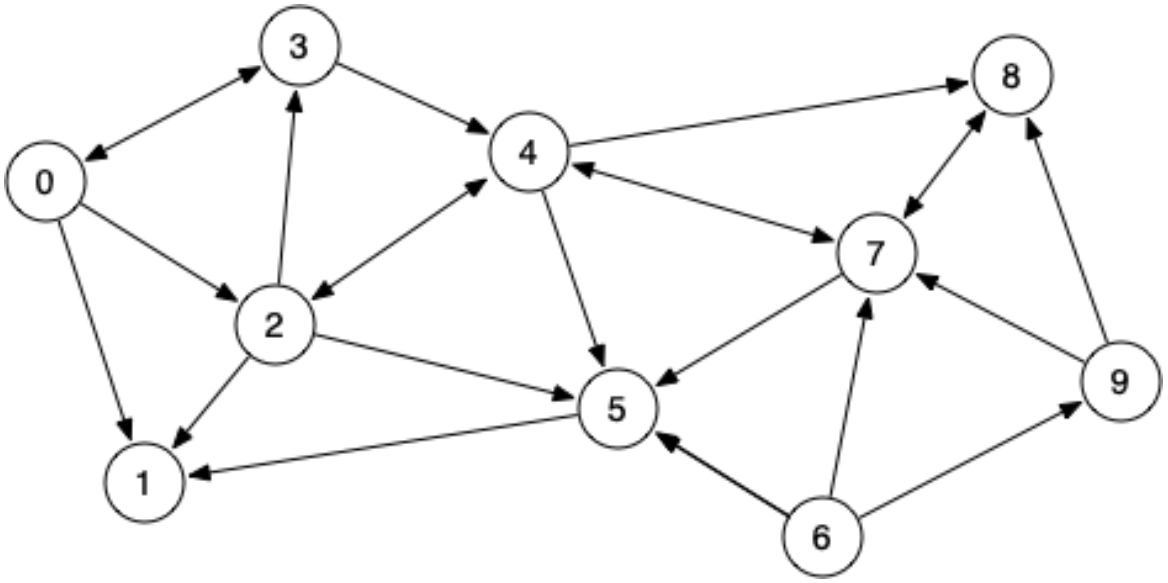
- To explore an application of weighted graphs and minimum spanning trees (In Tutorial)
- To gain a better understanding of minimum spanning trees and MST algorithms (In Tutorial)
- To see how graphs might be used with real-world data (Graphs in Tutorial)

▼ Major Announcements

- AT2 Released Wednesday Week 7 - Remember to keep working on it
  - Due 8pm Friday Week 10

▼ Reachability in Digraphs (Directed Graphs)

In the following graph:



▼ Which vertices are reachable from vertex 0?

0 1 2 3 4 5 7 8

▼ Which vertices are reachable from vertex 1?

1

▼ Which vertices are reachable from vertex 5?

1 5

▼ Which vertices are reachable from vertex 6?

0 1 2 3 4 5 6 7 8 9

Write a C function that takes a Graph and a starting Vertex and returns a set containing all of the vertices that can be reached by following a path from the starting point. Use the function template:

```
Set reachable(Graph g, Vertex src) { ... }
```

You may use any of the standard ADTs from the slides (e.g. Sets, Lists, Stacks, Queues).

▼ Answer 1

DFS (with stack instead) or BFS

```
Set reachable(Graph g, Vertex src) {
    Set seen = SetNew();

    Queue q = QueueNew();
    QueueEnqueue(q, src);
    SetAdd(seen, src);

    while (!QueueIsEmpty(q)) {
        Vertex v = QueueDequeue(q);

        for (Vertex w = 0; w < g->nV; w++) {
            if (g->edges[v][w] && !SetContains(seen,
w)) {
                QueueEnqueue(q, w);
                SetAdd(seen, w);
            }
        }
    }
    return seen;
}
```

▼ Answer 2

Recursive DFS, no stack or queue just uses a set adds it to the set and then checks recursively down each edge that the set contains.

```
Set reachable(Graph g, Vertex src) {
    Set seen = SetNew();
    doReachable(g, src, seen);
    return seen;
}

static void doReachable(Graph g, Vertex v, Set seen)
{
    SetAdd(seen, v);

    for (Vertex w = 0; w < g->nV; w++) {
        if (g->edges[v][w] && !SetContains(seen, w))
```

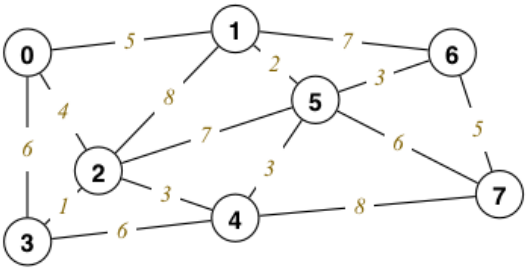
```
{
                doReachable(g, w, seen);
            }
        }
}
```

▼ Style Tip

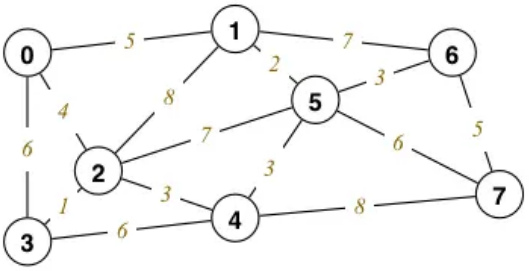
Remember helper functions should be made static.

▼ Dijkstra's Algorithm (Pass)

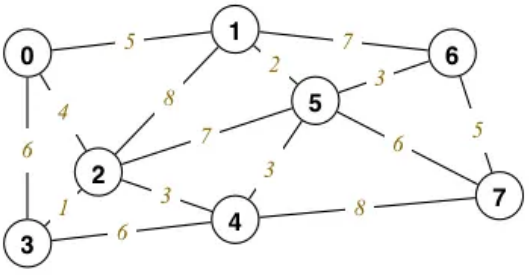
Trace the execution of Dijkstra's algorithm on the following graph to compute the minimum distances from source node 0 to all other vertices:



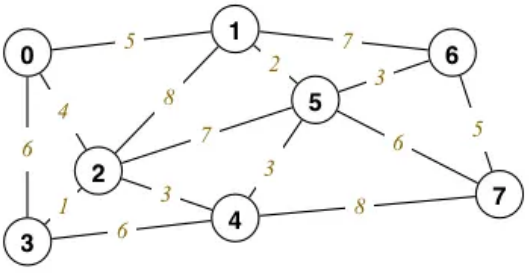
Show the values of vSet, dist[] and pred[] after each iteration.



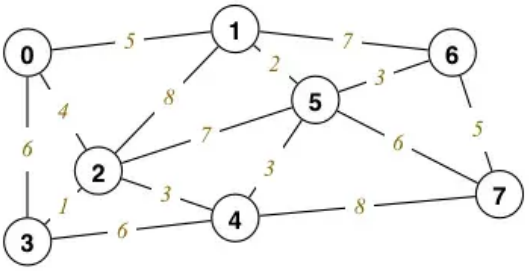
▼ How Does Dijkstra's Algorithm Work



- 1, Have the source vertex
- 2, Have all vertices bar the source in the VSET
- 3, Add the distance and predecessor from the current vertex (only if it is less altogether than the current distance).

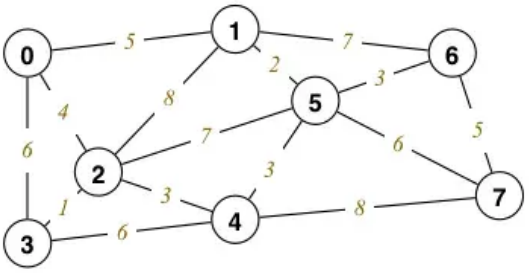


- Take the smallest distance vertex in the VSet
- Remove it from the VSet and continue from step 3 with this vertex instead of the source.



▼ Initialisation:

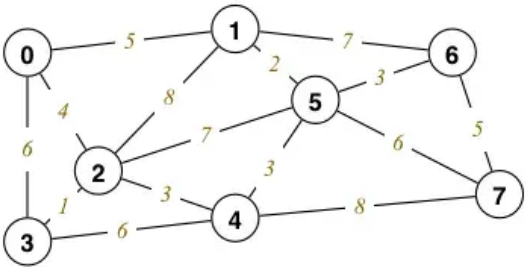
```
vSet = {  0,  1,  2,  3,  4,
5,  6,  7 }
dist = [  0,  ∞,  ∞,  ∞,  ∞,
∞,  ∞,  ∞ ]
pred = [ -1, -1, -1, -1, -1, -
1, -1, -1 ]
```

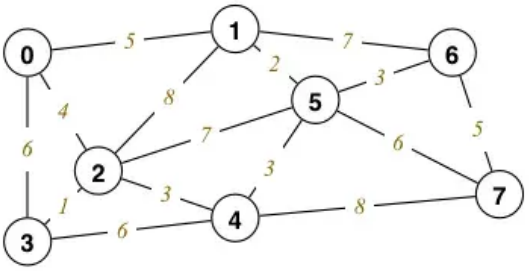


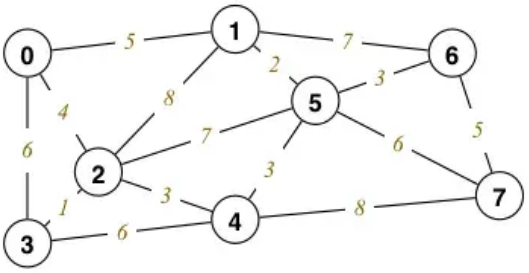
▼ Step 1 (What are the relations, what happens in vSet and what about predecessors for these)

The vertex in vSet with minimum dist[] is 0. Relaxation along the edges (0, 1, 5), (0, 2, 4) and (0, 3, 6) results in:

```
vSet = {      1,  2,  3,  4,
5,  6,  7 }
dist = [  0,  5,  4,  6,  ∞,
∞,  ∞,  ∞ ]
pred = [ -1,  0,  0,  0, -1, -
1, -1, -1 ]
```







▼ Step 2 (What are the relations, what

happens in vSet and what about predecessors for these)

Now the vertex in vSet with minimum dist[] is 2. Considering all edges from 2 to nodes still in vSet:

- relaxation along (2, 1, 8) does not give us a shorter distance to node 1
- relaxation along (2, 3, 1) yields a smaller value (4 + 1 = 5) for dist[3], and pred[3] is updated to 2
- relaxation along (2, 4, 3) yields a smaller value (4 + 3 = 7) for dist[4], and pred[4] is updated to 2
- relaxation along (2, 5, 7) yields a smaller value (4 + 7 = 11) for dist[5], and pred[5] is updated to 2

```
vSet = {      1,      3,  4,
5,  6,  7 }
dist = [  0,  5,  4,  5,  7, 1
1,  ∞,  ∞ ]
pred = [ -1,  0,  0,  2,  2,
2, -1, -1 ]
```

▼ Step 3 (What are the relations, what happens in vSet and what about

predecessors for these)

Next, we could choose either 1 or 3, since both vertices have minimum distance 5. Suppose we choose 1. Relaxation along (1, 5, 2) and (1, 6, 7) results in new values for nodes 5 and 6:

```
vSet = {              3,  4,
5,  6,  7 }
dist = [  0,  5,  4,  5,  7,
7, 12,  ∞ ]
pred = [ -1,  0,  0,  2,  2,
1,  1, -1 ]
```

▼ Step 4 (What are the relations, what happens in vSet and what about predecessors for these)

Now we consider vertex 3. The only adjacent node still in vSet is 4, but there is no shorter path to 4 through 3. Hence no update to dist[] or pred[]:

```
vSet = {                  4,
5,  6,  7 }
dist = [  0,  5,  4,  5,  7,
7, 12,  ∞ ]
pred = [ -1,  0,  0,  2,  2,
1,  1, -1 ]
```

▼ Step 5 (What are the relations, what happens in vSet and what about predecessors for these)

Next we could choose either vertex 4 or 5. Suppose we choose 4. Edge (4, 7, 8) is the only one that leads to an update:

```
vSet = {
5,  6,  7 }
dist = [  0,  5,  4,  5,  7,
7, 12, 15 ]
pred = [ -1,  0,  0,  2,  2,
1,  1,  4 ]
```

▼ Step 6 (What are the relations, what happens in vSet and what about predecessors for these)

Vertex 5 is next. Relaxation along edges (5, 6, 3) and (5, 7, 6) results in:

```
vSet = {
6,  7 }
dist = [  0,  5,  4,  5,  7,
7, 10, 13 ]
pred = [ -1,  0,  0,  2,  2,
1,  5,  5 ]
```

▼ Step 7 (What are the relations, what happens in vSet and what about predecessors for these)

Of the two vertices left in vSet, 6 has the shorter distance. Edge (6, 7, 5) does not update the values for node 7 since dist[7] = 13 < dist[6] + 5 = 15. Hence:

```
vSet = {
7 }
dist = [  0,  5,  4,  5,  7,
7, 10, 13 ]
pred = [ -1,  0,  0,  2,  2,
1,  5,  5 ]
```

▼ What are the least distances for each node (individual pass)?

Processing the last remaining vertex in vSet will obviously not change anything. The values in pred[] determine shortest paths to all nodes as follows:

```
0: distance =  0, shortest pat
h: 0
1: distance =  5, shortest pat
h: 0-1
2: distance =  4, shortest pat
h: 0-2
```

```
3: distance =  5, shortest pat
h: 0-2-3
4: distance =  7, shortest pat
h: 0-2-4
5: distance =  7, shortest pat
h: 0-1-5
6: distance = 10, shortest pat
h: 0-1-5-6
7: distance = 13, shortest pat
h: 0-1-5-7
```

▼ If time persists show with this graph visualiser by Jonah

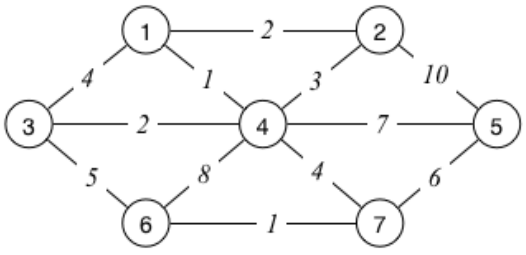
https://graphalgos.surge.sh/

▼ Kruskal's Algorithm

▼ What does Kruskal's Algorithm Do

Constructs MST by gradually joining the graph together in a forest, with each subgraph as a single node then adding new edges.

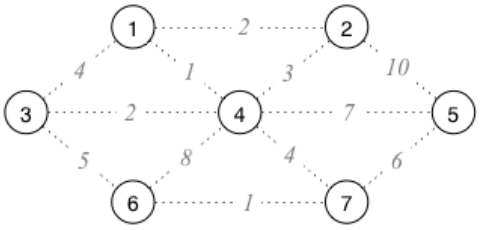
▼ Show how it would construct the MST for the graph below:



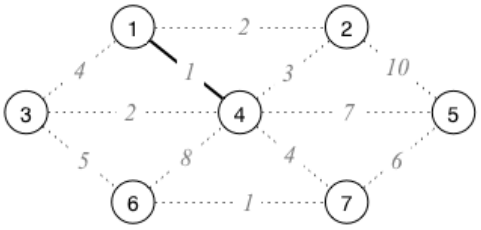
▼ No Edges

Kruskal's algorithm for finding a minimum spanning tree of a graph can be expressed as follows:

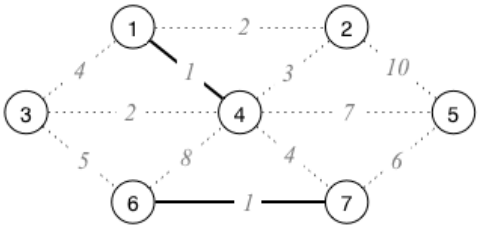
```
typedef Graph MSTre
e;

MSTree kruskalFindMS
T(Graph g) {
    MSTree mst = Gra
phNew(g->nV);// MST
initially empty Edge
eList[g->nE];// sort
ed array of edges
edges(eList, g->nE,
g);
```



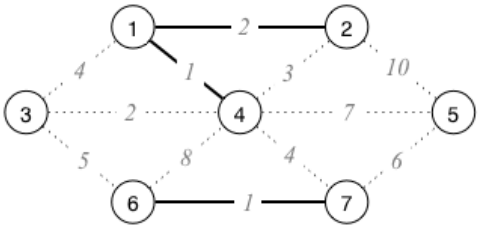
▼ **Next Edge (Assume choose smallest edge index)**



▼ **Next Edge (Assume choose smallest edge index)**



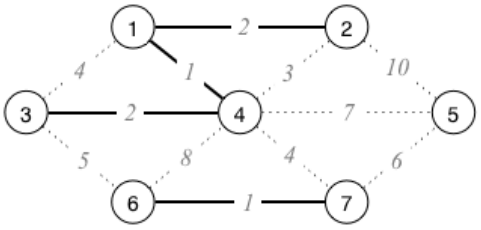
▼ **Next Edge (Assume choose smallest edge index)**



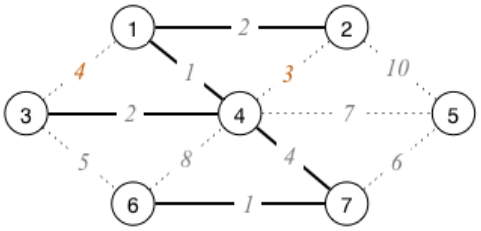

```
    sortEdgeList(eLi
st, g->nE);
    for (int i = 0;
mst->nE < g->nV - 1;
i++) {
        Edge e = eLi
st[i];
        GraphAddEdge
(mst, e);
            if (Grap
hHasCycle(mst)) Grap
hRemoveEdge(mst, e);
    }
    return mst;
}
```

```
typedef Graph MSTre
e;

MSTree kruskalFindMS
T(Graph g) {
    MSTree mst = Gra
phNew(g->nV);// MST
initially empty Edge
eList[g->nE];// sort
ed array of edges
edges(eList, g->nE,
g);
```

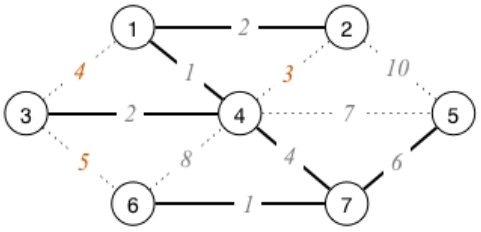
▼ **Next Edge (Assume choose smallest edge index)**



▼ **Next Edge (Assume choose smallest edge index)**



▼ **Next Edge (Assume choose smallest edge index)**



▼ **How many edges did we consider?**

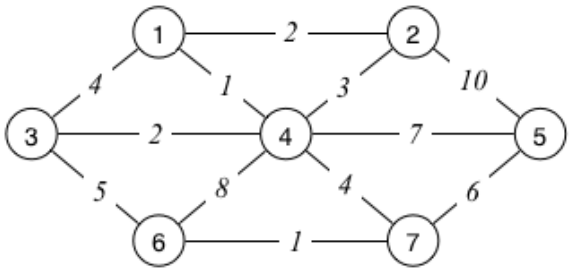
9 out of 12

```
    sortEdgeList(eList, g->nE);
    for (int i = 0; mst->nE < g->nV - 1; i++) {
        Edge e = eList[i];
        GraphAddEdge(mst, e);
            if (GraphHasCycle(mst)) GraphRemoveEdge(mst, e);
    }
    return mst;
}
```

```
typedef Graph MSTree;

MSTree kruskalFindMST(Graph g) {
    MSTree mst = GraphNew(g->nV);// MST initially empty Edge eList[g->nE];// sorted array of edges
```

▼ For a graph $G(V, E)$, what is the least number of edges we might need to consider? What is the most number of edges we might have to consider?

Least → V-1 Edges as that is the lowest amount of edges to connect each node, worst case is considering all E edges.




```
edges(eList, g->nE,
g);
    sortEdgeList(eLi
st, g->nE);
    for (int i = 0;
mst->nE < g->nV - 1;
i++) {
        Edge e = eLi
st[i];
        GraphAddEdge
(mst, e);
            if (Grap
hHasCycle(mst)) Grap
hRemoveEdge(mst, e);
    }
    return mst;
}
```

▼ Modify the graph above to force Kruskal's algorithm to the worst case.

Remove edges 4-5 and 5-7 (must consider all edges)

Add new vertex connected with weight 11.

▼ Prim's Algorithm

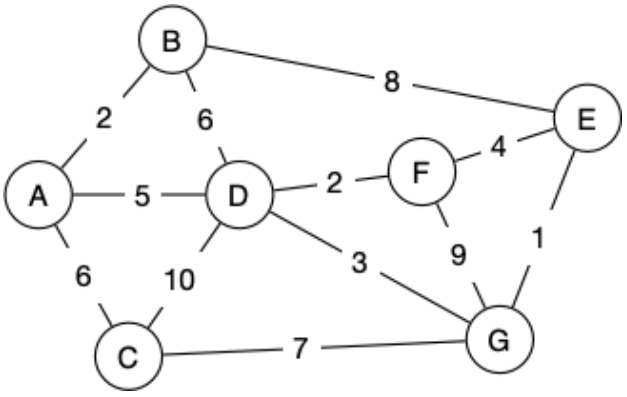
▼ What is Prim's algorithm abstractly?

1. Start from any vertex *v* and empty MST
2. Choose edge not already in MST, satisfying

- incident on a vertex *s* already in MST
- incident on a vertex *t* not already in MST
- with minimal weight of all such edges

3. Add chosen edge to MST
4. Repeat until MST covers all vertices

▼ How does Prim's algorithm create an MST with this graph?



▼ Start A Initialising

- The MST contains just the starting vertex: V = {A}, E = {}

▼ Iteration 1:

▼ Choices

- Choices: AB, AC, AD ... AB is cheapest

▼ Updates

- Updated MST: V = {A, B}, E = {AB}

▼ Iteration 2:

▼ Choices

- Choices: AC, AD, BD, BE ... AD is cheapest

▼ Updates

- Updated MST: V = {A, B, D}, E = {AB, AD}

▼ Iteration 3:

▼ Choices

- Choices: AC, BE, DC, DF, DG ... DF is cheapest

▼ Updates

- Updated MST: V = {A, B, D, F}, E = {AB, AD, DF}

▼ Iteration 4:

▼ Choices

- Choices: AC, BE, DC, DG, FE, FG ... DG is cheapest

▼ Updates

- Updated MST: V = {A, B, D, F, G}, E = {AB, AD, DF, DG}

▼ Iteration 5:

▼ Choices

- Choices: AC, BE, DC, FE, GC, GE ... GE is cheapest

▼ Updates

- Updated MST: V = {A, B, D, E, F, G}, E = {AB, AD, DF, DG, GE}

▼ Iteration 6:

▼ Choices

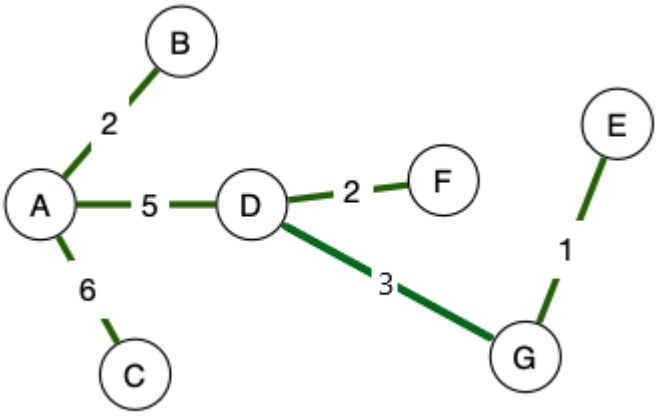
- Choices: AC, DC, GC ... AC is cheapest

▼ Updates

- Updated MST: V = {A, B, C, D, E, F, G}, E = {AB, AD, DF, DG, GE, AC}

All vertices are now included, so we have the MST:

▼ Final MST


B
2
E
A
5
D
2
F
1
6
3
C
G